

Rapport Complet : Algorithmes de Flot Maximal et Coût Minimal

Achour DJERADA

6 mai 2026

1 Introduction et Objectif du Projet

Ce projet implémente une bibliothèque complète de résolution de problèmes classiques de la recherche opérationnelle et de l'optimisation dans les graphes :

- **Flot maximal** : faire transiter un maximum d'unités de la source au puits.
- **Flot de coût minimal** : envoyer ce flot selon les chemins les moins chers.
- **Détection de cycles de coût négatif** dans le résiduel.

Contrainte : **aucune librairie de flots** (type NetworkX) n'est utilisée. Tout est codé à la main.

2 Format d'Entrée et Sorties Attendues

2.1 Format d'entrée

Le fichier d'entrée commence par une ligne :

```
#nodes #arcs s t
```

Puis `#arcs` lignes de la forme :

```
u v capacite cout_unitaire
```

Contraintes : `#nodes > 0, #arcs >= 0, 0 <= s, t < #nodes, capacite >= 0`.

2.2 Sorties

Le programme affiche :

- La valeur du flot maximal.
- La valeur du flot sur chaque arc (flot/capacité).
- La min cut : l'ensemble S , les arcs de coupe et la capacité totale.
- Le résultat min-cost max-flow via Bellman-Ford et via Dijkstra+potentiels.
- La présence ou non d'un cycle de coût négatif résiduel.

3 Structure de Données : Le Graphe Résiduel (src/graph.py)

Le graphe résiduel est au coeur du projet. Chaque arc original $u \rightarrow v$ a un arc inverse $v \rightarrow u$ de capacité initiale nulle et de coût négatif.

3.1 L'objet Edge

```
@dataclass
class Edge:
    to: int
    rev: int
    cap: int
    cost: int
    original_cap: int
    is_reverse: bool
```

rev pointe vers l'arc inverse dans la liste d'adjacence, ce qui permet les mises à jour en $\mathcal{O}(1)$ lors des augmentations de flot.

3.2 Pourquoi les listes d'adjacence ?

Les réseaux sont souvent creux ($E \ll V^2$). Les listes d'adjacence permettent un parcours local en $\mathcal{O}(\deg(u))$ et une mémoire en $\mathcal{O}(V + E)$.

4 Algorithmes et Complexités

4.1 Flot Maximal : Edmonds-Karp (src/max_flow.py)

On utilise une BFS pour trouver un chemin augmentant minimal en nombre d'arêtes.

- BFS : $\mathcal{O}(E)$.
- Nombre d'itérations : $\mathcal{O}(V \cdot E)$.
- Complexité totale : $\mathcal{O}(V \cdot E^2)$.

4.2 Min Cut

Après le flot maximal, la min cut est obtenue en faisant une BFS dans le résiduel : S est l'ensemble des sommets atteignables depuis la source, et les arcs de coupe sont les arcs originaux $u \in S, v \notin S$.

- Complexité : $\mathcal{O}(V + E)$.

4.3 Min-Cost Max-Flow par Bellman-Ford (src/min_cost_flow.py)

Les arcs inverses induisent des coûts négatifs, d'où l'usage de Bellman-Ford.

- Complexité : $\mathcal{O}(F \cdot V \cdot E)$.

4.4 Min-Cost Max-Flow avec Dijkstra + Potentiels

On calcule un potentiel initial via Bellman-Ford, puis on exécute Dijkstra sur les coûts réduits $c'(u, v) = c(u, v) + \pi(u) - \pi(v)$.

- Dijkstra : $\mathcal{O}(E \log V)$.
- Complexité totale : $\mathcal{O}(V \cdot E + F \cdot E \log V)$.

4.5 Détection de cycle négatif (src/cycle_detection.py)

On applique Bellman-Ford avec distances initiales à 0 (super-source implicite).

- Complexité : $\mathcal{O}(V \cdot E)$.

5 Tests

Les tests automatisés sont lancés via :

```
python tests/run_tests.py
```

Ils couvrent : flot max, min-cost, cycle négatif, labels Graphviz, et min cut.

6 Exemple d'Exécution

Commande :

```
python src/main.py data/exemple_entree.txt
```

Sorties attendues : flot maximal, min cut, min-cost (Bellman-Ford / Dijkstra), cycle négatif résiduel.

7 Conclusion

La structure résiduelle avec arcs inverses et index croisés offre des mises à jour rapides et un socle correct pour les algorithmes de flot. Le projet respecte les contraintes et fournit des résultats reproductibles.

8 Bibliographie

1. Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network Flows : Theory, Algorithms, and Applications*. Prentice Hall.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
3. Tarjan, R. E. (1983). *Data Structures and Network Algorithms*. SIAM.